



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 09

NOMBRE COMPLETO: Cordero Miranda Evelyn Patricia

N° de Cuenta: 317314975

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 06

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 23 de abril del 2026

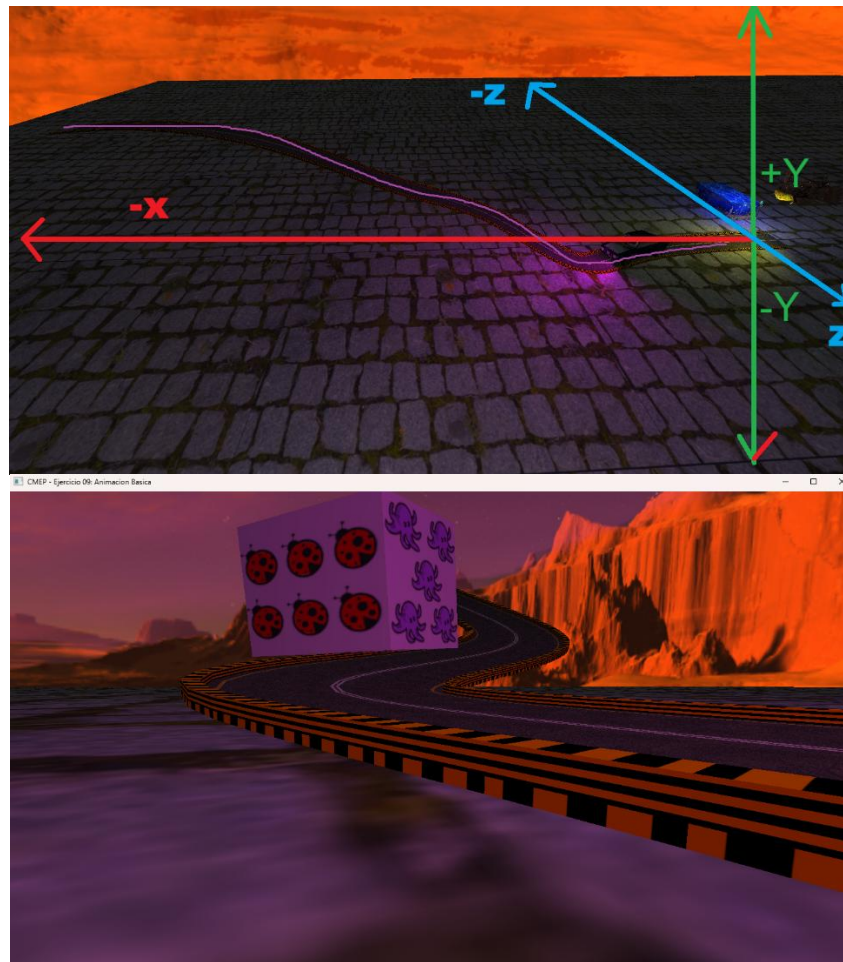
CALIFICACIÓN: _____

ACTIVIDADES A REALIZAR

RECORRIDO PISTA

INSTRUCCIÓN: “Hacer que su carro recorra la pista *Adecuadamente (curvas, inclinación en rampa, faro frontal ilumina la pista)* desde el inicio hasta el punto extremo y ahí se detenga. Se puede repetir la animación si se presiona una tecla designada.”

Primero explicaremos la lógica que se uso detrás para completar este recorrido, es importante iniciar mencionando que creamos una ruta de 17 etapas funcional que se uso para el coche y para la nave. En un inicio la propuesta se enfoco en ir haciendo la lógica de animación pingpong con un cubo e ir avanzando poco en el eje -X, poco en el eje $\pm Z$ y cuando fuera necesario poco en el eje +Y pero esta primera propuesta no fue la mejor, era muy tediosa, muy tardada y como era calculo sobre papel había muchos errores.



Sin embargo, se pudo rescatar algo importante, la estructura que usaríamos para nuestra animación se basaba en los siguientes condicionales:

```
else if (vehiculo-> etapa == X) {  
    // 1. DEFINIR EL DIVISOR (Distancia Total del Tramo en X)  
    distX = [Distancia absoluta entre la X actual y la X de la meta];  
  
    // 2. MOVIMIENTO (Proporciones)  
    // El avance en X es constante. Y y Z avanzan a un ritmo calculado para llegar al mismo tiempo.  
    vehiculo->posX -= velocidadBase * deltaTime;  
    vehiculo->posY += (velocidadBase * ((Y_Destino - Y_Origen) / distX)) * deltaTime;
```

```

vehiculo->posZ += (velocidadBase * ((Z_Destino - Z_Origen) / distX)) * deltaTime;

// 3. ROTACIÓN (Proporciones)
// Los ángulos rotan calculando la diferencia entre el ángulo final y el inicial.
vehiculo->anguloX += (velocidadBase * ((AngX_Destino - AngX_Origen) / distX)) * deltaTime;
vehiculo->anguloY += (velocidadBase * ((AngY_Destino - AngY_Origen) / distX)) * deltaTime;
vehiculo->anguloZ += (velocidadBase * ((AngZ_Destino - AngZ_Origen) / distX)) * deltaTime;

// 4. VERIFICACIÓN DE LÍMITES (Ida y Vuelta)
if (!vehiculo->deRegreso) {
    // --- MODO IDA ---
    if (vehiculo->posX <= X_Destino) {
        // ANCLAJE: Forzamos los valores exactos de la meta para evitar errores de decimales
        vehiculo->posX = X_Destino; vehiculo->posY = Y_Destino; vehiculo->posZ = Z_Destino;
        vehiculo->anguloX = AngX_Destino; /* ... (igual para Y y Z) ... */

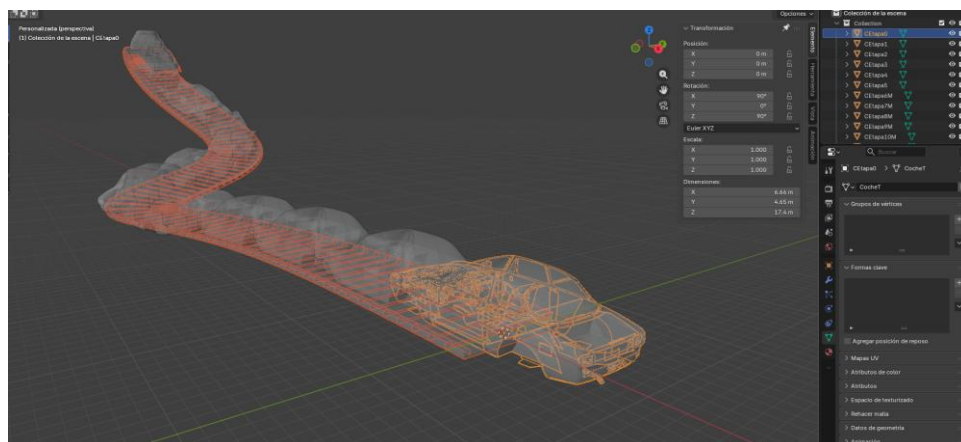
        // TRANSICIÓN: Avanzamos a la siguiente etapa
        vehiculo->etapa = X + 1;
    }
}
else {
    // --- MODO VUELTA (Reversa) ---
    if (vehiculo->posX >= X_Origen) {
        // ANCLAJE: Forzamos los valores exactos del origen
        vehiculo->posX = X_Origen; vehiculo->posY = Y_Origen; vehiculo->posZ = Z_Origen;
        vehiculo->anguloX = AngX_Origen; /* ... (igual para Y y Z) ... */

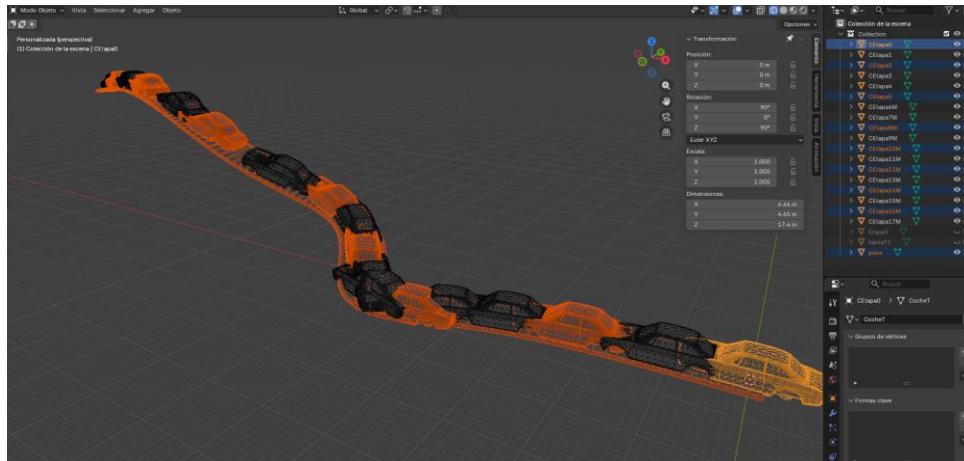
        // TRANSICIÓN: Retrocedemos a la etapa anterior
        vehiculo->etapa = X - 1;
    }
}
}
}

```

Definimos esta estructura condicional para cada etapa como el motor de nuestras pequeñas ‘animaciones’, lo que hacemos es tomar el avance constante en el eje X como referencia base para calcular la tasa de cambio exacta de las demás coordenadas (Y, Z) y de las rotaciones, asegurando que todas las transformaciones se completen simultáneamente al finalizar el tramo, esto para evitar que fuéramos avanzando de uno en uno y llegáramos primero a Y antes que a X. Además, integramos un sistema de ‘anclaje’ de límites que corrige las desviaciones acumuladas por errores de punto flotante debido a nuestras operaciones, garantizando que el vehículo que este haciendo el recorrido (ya sea el coche o la nave) pudiera tener la posición y rotación correctas antes de transicionar a la siguiente etapa, ya sea de ida o en reversa (esta última aplicable más adelante).

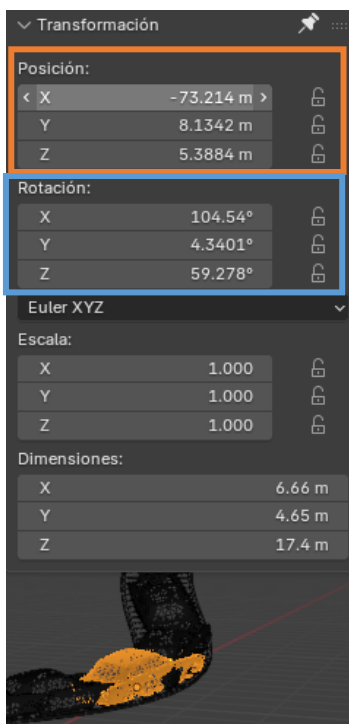
Ahora ¿Qué tuco usamos para calcular las posiciones? Era más fácil usar blender, exportando la pista base y también nuestro modelo de coche (se nos paso poner las llantas por lo que estas transformaciones las subiremos todas una unidad en +Y ya en OpenGL) así duplicábamos nuestro modelo, rotábamos y trasladábamos a lo largo de la pista a nuestro antojo, para obtener un total de 17 etapas (contando desde 0) como “puntos de control”.





Ejemplo de vaciado en etapa 9

En Blender



En Código

// --- ETAPA 9 ---

```
else if (v->etapa == 9) {
    distX = 8.773f;
    v->posX -= velTramoX * dtV;
    v->posY += (velTramoX * ((4.3884f - 1.9633f) / distX)) * dtV;
    v->posZ += (velTramoX * ((-8.1342f - 0.59452f) / distX)) * dtV;
    v->anguloX += (velTramoX * ((75.46f - 80.775f) / distX)) * dtV;
    v->anguloY += (velTramoX * ((59.278f - 42.484f) / distX)) * dtV;
    v->anguloZ += (velTramoX * ((4.3401f - 6.8825f) / distX)) * dtV;

    if (!v->deRegreso) {
        if (v->posX <= -73.214f) {
            v->posX = -73.214f; v->posY = 4.3884f; v->posZ = -8.1342f;
            v->anguloX = 75.46f; v->anguloY = 59.278f; v->anguloZ = 4.3401f;
            v->etapa = 10;
        }
    }
    else {
        if (v->posX >= -64.441f) {
            v->posX = -64.441f; v->posY = 1.9633f; v->posZ = 0.59452f;
            v->anguloX = 80.775f; v->anguloY = 42.484f; v->anguloZ = 6.8825f;
            v->etapa = 8;
        }
    }
}
```

Consideraciones:

- Recordemos que a Y le restamos 1 unidad
 $v->posY += (velTramoX * ((4.3884f - 1.9633f) / distX)) * dtV;$
- Los valores en verde son los de la transformación pasada (en este caso etapa 8)
- $distX = 8.773f$; viene de la resta $X_{destino} - X_{origen}$ (Para este caso sería $73.214 - 64.441 = 8.773$)
- El valor de $75.46f$ y $80.775f$ se colocan así por la diferencia de 180° que evita que nuestro coche este volteado (Para este caso $180 - 104.54 = 75.46f$)

Este mismo proceso anterior se repite para cada una de las etapas con las transformaciones dadas por blender, ahora, ya que este vaciado resulto en un código inmenso de líneas dentro de nuestro main, decidimos separarlo haciendo su propia clase de animación de pista de recorrido. Por lo cual, ahora si podemos pasar a explicar como fueron los cambios que tuvimos en nuestros archivos.

RecorridoP.h

En un nuevo header encapsulamos toda la lógica de la animación como lo solemos hacer en materias como POO (Programación orientada a objetos). Este header contendrá dos puntos importantes, la instancia de una estructura que nombraremos Vehiculo, que actuará como una

unidad de memoria para cada modelo, ya sea nuestro coche, nave o incluso alguna maquina a vapor. Por otro lado, se definió la clase RecorridoP, la cual instancia a los vehículos y centraliza nuestro motor matemático a través de su método actualizarRuta(). Dicho método puede recibir deltaTime para que nuestra velocidad de movimiento de la animación sea independiente del procesador, velTramoX como escalar de la velocidad del escenario, y una referencia de memoria (bool& reiniciaCoche) que permite establecer un paso por referencia que más adelante se usa para reiniciar el recorrido del coche.

```
#pragma once
#include<stdio.h>
#include<glew.h>
#include<glfw3.h>

// Cada estructura "vehiculo" esta compuesta por..."
struct Vehiculo {
    // Coordenadas de posición en nuestro escenario
    float posX;
    float posY;
    float posZ;
    // Rotaciones de Euler para alinear nuestros modelos con la pista
    float anguloX;
    float anguloY;
    float anguloZ;
    int etapa; // Numero de etapa (parte de recorrido) donde estamos
    bool deRegreso; // ¿Vamos de 0 -> -x o de -x -> 0?
    bool termino; // ¿Llegamos a la meta?
};

// =====
// CLASE: RecorridoP (Gestor de la Animación)
// =====
class RecorridoP {
public:

    // --- ATRIBUTOS PÚBLICOS ---
    // Para empaquetar otros modelos que igual ocupen animarse :3
    Vehiculo coche;
    Vehiculo nave;

    // --- MÉTODOS ---
    // 1. Constructor
    RecorridoP();

    // 2. Función de Actualización:
    // Recibe 3 parámetros necesarios para funcionar:
    // - deltaTime: Para que el movimiento sea fluido sin importar los FPS de la computadora.
    // - velTramoX: La velocidad base del recorrido (ya que X va a ser marcador de distancia)
    // - reiniciaCoche (bool&): La referencia a la tecla 'M'
    void actualizarRuta(float deltaTime, float velTramoX, bool& reiniciaCoche);
    // & -> Paso por referencia... Reseteo del coche
};
```

RecorridoP.cpp

La primera sección de la función actualizarRuta se encarga de preparar a los vehículos mediante un arreglo de punteros, iterando así entre nuestro coche y nave. También se controla el reinicio del recorrido (tecla 'M'). Sin embargo, la variable más crucial es dtV, misma que nos ayuda a evaluar si el vehículo va de regreso; si es así, invierte el signo del deltaTime (-deltaTime). Este nos permite correr el tiempo hacia atrás para que nuestro vehículo recorra la pista en reversa utilizando exactamente las mismas etapas.

```
#include "RecorridoP.h"

// CONSTRUCTOR: Valores Iniciales
RecorridoP::RecorridoP() {
```

```

// INICIALIZACION COCHE Y NAVE:
// Orden: posX, posY, posZ, anguloX, anguloY, anguloZ, etapa, deRegreso, termino
coche = { 0.0f, -1.0f, 0.0f, 90.0f, 90.0f, 0.0f, 1, false, false };
nave = { -177.46f, 25.389f, -23.568f, 96.958f, 70.654f, 1.6187f, 17, true, false };
}

// FUNCIÓN PRINCIPAL: Actualización de la Máquina de Estados
void RecorridoP::actualizarRuta(float deltaTime, float velTramoX, bool& reiniciaCocher) {

    // 1. Empaquetamos los vehículos en un arreglo de punteros.
    // Al usar punteros (&), lo que modifiquemos aquí afectará a las variables originales.
    Vehiculo* vehiculos[2] = { &coche, &nave };

    // 2. DECLARACIÓN DE VARIABLES DE APOYO (Reutilizables en cada ciclo del for)
    int i_vehiculo;           // 0: Primera vuelta - 1: Segunda vuelta
    Vehiculo* v;              // Coche o Nave
    float dtV;                // Deltatime V (+deltatime: Ida - -deltatime: regreso)
    float distX;              // Guardamos el tramo en X a recorrer

    // 3. CICLO: Primero el coche (0) y luego la nave (1)
    for (i_vehiculo = 0; i_vehiculo < 2; i_vehiculo++) {
        v = vehiculos[i_vehiculo]; // 'v' ahora apunta al vehículo en turno

        // ¿El vehículo ya llegó a su destino y está apagado?
        if (v->termino) {

            // Si es el coche(0) y se presiona la tecla 'M'(reiniciaCocher=true)
            if (i_vehiculo == 0 && reiniciaCocher) {
                v->termino = false; // Le quitamos la pausa
                v->deRegreso = false; // Lo ponemos en dirección de ida
                v->etapa = 1; // Lo reseteamos a la etapa 1

                // Apagamos gracias al paso por referencia (&)
                reiniciaCocher = false;
            }
            else {
                // Si es la nave o no apretamos 'M' usamos 'continue'
                continue;
            }
        }

        // TRUCO EN Ping-Pong
        // Si es TRUE, el tiempo corre hacia atrás (-deltaTime).
        // Si es FALSE, el tiempo corre normal (deltaTime).
        dtV = v->deRegreso ? -deltaTime : deltaTime;
    }
}

```

Después tenemos nuestras 17 etapas de animación mediante una Máquina de estados finita, con la que cada condicional nos representa un pequeño tramo específico del recorrido por la pisa

```

// =====
// ETAPA 1: Inicio del recorrido (Matemática Base)
// =====
if (v->etapa == 1) {
    // distX: Distancia total a recorrer en el eje X durante esta etapa.
    // Sirve como divisor para calcular las proporciones exactas de Y, Z y ángulos.
    distX = 10.544f;

    // --- Movimiento ---
    // El vehículo avanza a velocidad constante en X (velTramoX).
    v->posX += velTramoX * dtV;

    // Para Y y Z calculamos la proporción: (Destino - Origen) / distX.
    // Al multiplicar esto por velTramoX, obligamos a que alcancen su destino a la par.
    v->posY += (velTramoX * ((-1.0f - -1.0f) / distX)) * dtV;
    v->posZ += (velTramoX * ((0.61976f - 0.0f) / distX)) * dtV;

    // --- Rotación ---
    // Misma lógica matemática para que el modelo rote suavemente mientras avanza.
    v->anguloX += (velTramoX * ((90.0f - 90.0f) / distX)) * dtV;
    v->anguloY += (velTramoX * ((93.59f - 90.0f) / distX)) * dtV;
    v->anguloZ += (velTramoX * ((0.0f - 0.0f) / distX)) * dtV;

    if (!v->deRegreso) {
        // --- MODO IDA ---
        // Si rebasamos la meta en X (<= porque avanzamos hacia los negativos), anclamos
        // los valores exactos y pasamos a la siguiente etapa.
        if (v->posX <= -10.544f) {
            v->posX = -10.544f; v->posY = -1.0f; v->posZ = 0.61976f;
            v->anguloX = 90.0f; v->anguloY = 93.59f; v->anguloZ = 0.0f;
            v->etapa = 2;
        }
    }
}

```



```

    }
    else {
        // --- MODO VUELTA (Reversa) ---
        // Si viajamos en reversa y llegamos al origen (0.0), detenemos el vehículo.
        if (v->posX >= 0.0f) {
            v->posX = 0.0f; v->posY = -1.0f; v->posZ = 0.0f;
            v->anguloX = 90.0f; v->anguloY = 90.0f; v->anguloZ = 0.0f;
            v->termino = true; // Ambos V entran en pausa al llegar al inicio
        }
    }
}

// =====
// ETAPA 2:
// =====
else if (v->etapa == 2) {
    distX = 13.056f;
    v->posX -= velTramoX * dtV;
    // calculamos la proporción: (Destino - Origen) / distX.
    v->posY += (velTramoX * ((-1.0f - -1.0f) / distX)) * dtV;
    v->posZ += (velTramoX * ((1.4759f - 0.61976f) / distX)) * dtV;
    v->anguloX += (velTramoX * ((90.0f - 90.0f) / distX)) * dtV;
    v->anguloY += (velTramoX * ((98.094f - 93.59f) / distX)) * dtV;
    v->anguloZ += (velTramoX * ((0.0f - 0.0f) / distX)) * dtV;

    if (!v->deRegreso) {
        if (v->posX <= -23.6f) { // Meta de Etapa 2
            v->posX = -23.6f; v->posY = -1.0f; v->posZ = 1.4759f;
            v->anguloX = 90.0f; v->anguloY = 98.094f; v->anguloZ = 0.0f;
            v->etapa = 3;
        }
    }
    else {
        if (v->posX >= -10.544f) { // Origen de Etapa 2 (Para la vuelta)
            v->posX = -10.544f; v->posY = -1.0f; v->posZ = 0.61976f;
            v->anguloX = 90.0f; v->anguloY = 93.59f; v->anguloZ = 0.0f;
            v->etapa = 1;
        }
    }
}

// --- ETAPA 3 ---
else if (v->etapa == 3) {
    distX = 10.437f;
    v->posX -= velTramoX * dtV;
    v->posY += (velTramoX * ((-1.0f - -1.0f) / distX)) * dtV;
    v->posZ += (velTramoX * ((3.3737f - 1.4759f) / distX)) * dtV;
    v->anguloX += (velTramoX * ((90.0f - 90.0f) / distX)) * dtV;
    v->anguloY += (velTramoX * ((104.08f - 98.094f) / distX)) * dtV;
    v->anguloZ += (velTramoX * ((0.0f - 0.0f) / distX)) * dtV;

    if (!v->deRegreso) {
        if (v->posX <= -34.037f) {
            v->posX = -34.037f; v->posY = -1.0f; v->posZ = 3.3737f;
            v->anguloX = 90.0f; v->anguloY = 104.08f; v->anguloZ = 0.0f;
            v->etapa = 4;
        }
    }
    else {
        if (v->posX >= -23.6f) {
            v->posX = -23.6f; v->posY = -1.0f; v->posZ = 1.4759f;
            v->anguloX = 90.0f; v->anguloY = 98.094f; v->anguloZ = 0.0f;
            v->etapa = 2;
        }
    }
}

// --- ETAPA 4 ---
else if (v->etapa == 4) {
    distX = 9.383f;
    v->posX -= velTramoX * dtV;
    v->posY += (velTramoX * ((-1.0f - -1.0f) / distX)) * dtV;
    v->posZ += (velTramoX * ((6.5839f - 3.3737f) / distX)) * dtV;
    v->anguloX += (velTramoX * ((90.0f - 90.0f) / distX)) * dtV;
    v->anguloY += (velTramoX * ((113.53f - 104.08f) / distX)) * dtV;
    v->anguloZ += (velTramoX * ((0.0f - 0.0f) / distX)) * dtV;

    if (!v->deRegreso) {
        if (v->posX <= -43.42f) {
            v->posX = -43.42f; v->posY = -1.0f; v->posZ = 6.5839f;
            v->anguloX = 90.0f; v->anguloY = 113.53f; v->anguloZ = 0.0f;
            v->etapa = 5;
        }
    }
    else {
        if (v->posX >= -34.037f) {
            v->posX = -34.037f; v->posY = -1.0f; v->posZ = 3.3737f;
            v->anguloX = 90.0f; v->anguloY = 104.08f; v->anguloZ = 0.0f;
            v->etapa = 3;
        }
    }
}
}

```

```

// --- ETAPA 5 ---
else if (v->etapa == 5) {
    distX = 4.644f;
    v->posX -= velTramoX * dtV;
    v->posY += (velTramoX * ((-1.0f - -1.0f) / distX)) * dtV;
    v->posZ += (velTramoX * ((9.4087f - 6.5839f) / distX)) * dtV;
    v->anguloX += (velTramoX * ((90.0f - 90.0f) / distX)) * dtV;
    v->anguloY += (velTramoX * ((97.444f - 113.53f) / distX)) * dtV;
    v->anguloZ += (velTramoX * ((0.0f - 0.0f) / distX)) * dtV;

    if (!v->deRegreso) {
        if (v->posX <= -48.064f) {
            v->posX = -48.064f; v->posY = -1.0f; v->posZ = 9.4087f;
            v->anguloX = 90.0f; v->anguloY = 97.444f; v->anguloZ = 0.0f;
            v->etapa = 6;
        }
    }
    else {
        if (v->posX >= -43.42f) {
            v->posX = -43.42f; v->posY = -1.0f; v->posZ = 6.5839f;
            v->anguloX = 90.0f; v->anguloY = 113.53f; v->anguloZ = 0.0f;
            v->etapa = 4;
        }
    }
}

// --- ETAPA 6 ---
else if (v->etapa == 6) {
    distX = 6.776f;
    v->posX -= velTramoX * dtV;
    v->posY += (velTramoX * ((0.4627f - -1.0f) / distX)) * dtV;
    v->posZ += (velTramoX * ((9.4087f - 9.4087f) / distX)) * dtV;
    v->anguloX += (velTramoX * ((83.506f - 90.0f) / distX)) * dtV;
    v->anguloY += (velTramoX * ((77.925f - 97.444f) / distX)) * dtV;
    v->anguloZ += (velTramoX * ((1.3952f - 0.0f) / distX)) * dtV;

    if (!v->deRegreso) {
        if (v->posX <= -54.84f) {
            v->posX = -54.84f; v->posY = 0.4627f; v->posZ = 9.4087f;
            v->anguloX = 83.506f; v->anguloY = 77.925f; v->anguloZ = 1.3952f;
            v->etapa = 7;
        }
    }
    else {
        if (v->posX >= -48.064f) {
            v->posX = -48.064f; v->posY = -1.0f; v->posZ = 9.4087f;
            v->anguloX = 90.0f; v->anguloY = 97.444f; v->anguloZ = 0.0f;
            v->etapa = 5;
        }
    }
}

// --- ETAPA 7 ---
else if (v->etapa == 7) {
    distX = 1.907f;
    v->posX -= velTramoX * dtV;
    v->posY += (velTramoX * ((0.7826f - 0.4627f) / distX)) * dtV;
    v->posZ += (velTramoX * ((7.7146f - 9.4087f) / distX)) * dtV;
    v->anguloX += (velTramoX * ((83.506f - 83.506f) / distX)) * dtV;
    v->anguloY += (velTramoX * ((42.135f - 77.925f) / distX)) * dtV;
    v->anguloZ += (velTramoX * ((1.3952f - 1.3952f) / distX)) * dtV;

    if (!v->deRegreso) {
        if (v->posX <= -56.747f) {
            v->posX = -56.747f; v->posY = 0.7826f; v->posZ = 7.7146f;
            v->anguloX = 83.506f; v->anguloY = 42.135f; v->anguloZ = 1.3952f;
            v->etapa = 8;
        }
    }
    else {
        if (v->posX >= -54.84f) {
            v->posX = -54.84f; v->posY = 0.4627f; v->posZ = 9.4087f;
            v->anguloX = 83.506f; v->anguloY = 77.925f; v->anguloZ = 1.3952f;
            v->etapa = 6;
        }
    }
}

// --- ETAPA 8 ---
else if (v->etapa == 8) {
    distX = 7.694f;
    v->posX -= velTramoX * dtV;
    v->posY += (velTramoX * ((1.9633f - 0.7826f) / distX)) * dtV;
    v->posZ += (velTramoX * ((0.59452f - 7.7146f) / distX)) * dtV;
    v->anguloX += (velTramoX * ((80.775f - 83.506f) / distX)) * dtV;
    v->anguloY += (velTramoX * ((42.484f - 42.135f) / distX)) * dtV;
    v->anguloZ += (velTramoX * ((6.8825f - 1.3952f) / distX)) * dtV;

    if (!v->deRegreso) {
        if (v->posX <= -64.441f) {
            v->posX = -64.441f; v->posY = 1.9633f; v->posZ = 0.59452f;
            v->anguloX = 80.775f; v->anguloY = 42.484f; v->anguloZ = 6.8825f;

```



```

        v->etapa = 9;
    }
}
else {
    if (v->posX >= -56.747f) {
        v->posX = -56.747f; v->posY = 0.7826f; v->posZ = 7.7146f;
        v->anguloX = 83.506f; v->anguloY = 42.135f; v->anguloZ = 1.3952f;
        v->etapa = 7;
    }
}
}

// --- ETAPA 9 ---
else if (v->etapa == 9) {
    distX = 8.773f;
    v->posX -= velTramoX * dtV;
    v->posY += (velTramoX * ((4.3884f - 1.9633f) / distX)) * dtV;
    v->posZ += (velTramoX * ((-8.1342f - 0.59452f) / distX)) * dtV;
    v->anguloX += (velTramoX * ((75.46f - 80.775f) / distX)) * dtV;
    v->anguloY += (velTramoX * ((59.278f - 42.484f) / distX)) * dtV;
    v->anguloZ += (velTramoX * ((4.3401f - 6.8825f) / distX)) * dtV;

    if (!v->deRegreso) {
        if (v->posX <= -73.214f) {
            v->posX = -73.214f; v->posY = 4.3884f; v->posZ = -8.1342f;
            v->anguloX = 75.46f; v->anguloY = 59.278f; v->anguloZ = 4.3401f;
            v->etapa = 10;
        }
    }
    else {
        if (v->posX >= -64.441f) {
            v->posX = -64.441f; v->posY = 1.9633f; v->posZ = 0.59452f;
            v->anguloX = 80.775f; v->anguloY = 42.484f; v->anguloZ = 6.8825f;
            v->etapa = 8;
        }
    }
}

// --- ETAPA 10 ---
else if (v->etapa == 10) {
    distX = 7.314f;
    v->posX -= velTramoX * dtV;
    v->posY += (velTramoX * ((6.7213f - 4.3884f) / distX)) * dtV;
    v->posZ += (velTramoX * ((-10.532f - -8.1342f) / distX)) * dtV;
    v->anguloX += (velTramoX * ((69.25f - 75.46f) / distX)) * dtV;
    v->anguloY += (velTramoX * ((80.778f - 59.278f) / distX)) * dtV;
    v->anguloZ += (velTramoX * ((-2.4842f - 4.3401f) / distX)) * dtV;

    if (!v->deRegreso) {
        if (v->posX <= -80.528f) {
            v->posX = -80.528f; v->posY = 6.7213f; v->posZ = -10.532f;
            v->anguloX = 69.25f; v->anguloY = 80.778f; v->anguloZ = -2.4842f;
            v->etapa = 11;
        }
    }
    else {
        if (v->posX >= -73.214f) {
            v->posX = -73.214f; v->posY = 4.3884f; v->posZ = -8.1342f;
            v->anguloX = 75.46f; v->anguloY = 59.278f; v->anguloZ = 4.3401f;
            v->etapa = 9;
        }
    }
}

// --- ETAPA 11 ---
else if (v->etapa == 11) {
    distX = 12.931f;
    v->posX -= velTramoX * dtV;
    v->posY += (velTramoX * ((12.137f - 6.7213f) / distX)) * dtV;
    v->posZ += (velTramoX * ((-8.2756f - -10.532f) / distX)) * dtV;
    v->anguloX += (velTramoX * ((67.14f - 69.25f) / distX)) * dtV;
    v->anguloY += (velTramoX * ((105.3f - 80.778f) / distX)) * dtV;
    v->anguloZ += (velTramoX * ((-12.828f - -2.4842f) / distX)) * dtV;

    if (!v->deRegreso) {
        if (v->posX <= -93.459f) {
            v->posX = -93.459f; v->posY = 12.137f; v->posZ = -8.2756f;
            v->anguloX = 67.14f; v->anguloY = 105.3f; v->anguloZ = -12.828f;
            v->etapa = 12;
        }
    }
    else {
        if (v->posX >= -80.528f) {
            v->posX = -80.528f; v->posY = 6.7213f; v->posZ = -10.532f;
            v->anguloX = 69.25f; v->anguloY = 80.778f; v->anguloZ = -2.4842f;
            v->etapa = 10;
        }
    }
}

// --- ETAPA 12 ---
else if (v->etapa == 12) {
    distX = 17.811f;

```

```

v->posX -= velTramoX * dtV;
v->posY += (velTramoX * ((20.323f - 12.137f) / distX)) * dtV;
v->posZ += (velTramoX * ((-5.2718f - -8.2756f) / distX)) * dtV;
v->anguloX += (velTramoX * ((68.94f - 67.14f) / distX)) * dtV;
v->anguloY += (velTramoX * ((91.386f - 105.3f) / distX)) * dtV;
v->anguloZ += (velTramoX * ((-2.1826f - -12.828f) / distX)) * dtV;

if (!v->deRegreso) {
    if (v->posX <= -111.27f) {
        v->posX = -111.27f; v->posY = 20.323f; v->posZ = -5.2718f;
        v->anguloX = 68.94f; v->anguloY = 91.386f; v->anguloZ = -2.1826f;
        v->etapa = 13;
    }
}
else {
    if (v->posX >= -93.459f) {
        v->posX = -93.459f; v->posY = 12.137f; v->posZ = -8.2756f;
        v->anguloX = 67.14f; v->anguloY = 105.3f; v->anguloZ = -12.828f;
        v->etapa = 11;
    }
}
}

// --- ETAPA 13 ---
else if (v->etapa == 13) {
    distX = 11.11f;
    v->posX -= velTramoX * dtV;
    v->posY += (velTramoX * ((24.268f - 20.323f) / distX)) * dtV;
    v->posZ += (velTramoX * ((-6.3708f - -5.2718f) / distX)) * dtV;
    v->anguloX += (velTramoX * ((77.82f - 68.94f) / distX)) * dtV;
    v->anguloY += (velTramoX * ((76.554f - 91.386f) / distX)) * dtV;
    v->anguloZ += (velTramoX * ((-0.5779f - -2.1826f) / distX)) * dtV;

    if (!v->deRegreso) {
        if (v->posX <= -122.38f) {
            v->posX = -122.38f; v->posY = 24.268f; v->posZ = -6.3708f;
            v->anguloX = 77.82f; v->anguloY = 76.554f; v->anguloZ = -0.5779f;
            v->etapa = 14;
        }
    }
    else {
        if (v->posX >= -111.27f) {
            v->posX = -111.27f; v->posY = 20.323f; v->posZ = -5.2718f;
            v->anguloX = 68.94f; v->anguloY = 91.386f; v->anguloZ = -2.1826f;
            v->etapa = 12;
        }
    }
}

// --- ETAPA 14 ---
else if (v->etapa == 14) {
    distX = 20.11f;
    v->posX -= velTramoX * dtV;
    v->posY += (velTramoX * ((27.339f - 24.268f) / distX)) * dtV;
    v->posZ += (velTramoX * ((-11.258f - -6.3708f) / distX)) * dtV;
    v->anguloX += (velTramoX * ((86.84f - 77.82f) / distX)) * dtV;
    v->anguloY += (velTramoX * ((70.695f - 76.554f) / distX)) * dtV;
    v->anguloZ += (velTramoX * ((0.89183f - -0.5779f) / distX)) * dtV;

    if (!v->deRegreso) {
        if (v->posX <= -142.49f) {
            v->posX = -142.49f; v->posY = 27.339f; v->posZ = -11.258f;
            v->anguloX = 86.84f; v->anguloY = 70.695f; v->anguloZ = 0.89183f;
            v->etapa = 15;
        }
    }
    else {
        if (v->posX >= -122.38f) {
            v->posX = -122.38f; v->posY = 24.268f; v->posZ = -6.3708f;
            v->anguloX = 77.82f; v->anguloY = 76.554f; v->anguloZ = -0.5779f;
            v->etapa = 13;
        }
    }
}

// --- ETAPA 15 ---
else if (v->etapa == 15) {
    distX = 11.98f;
    v->posX -= velTramoX * dtV;
    v->posY += (velTramoX * ((27.491f - 27.339f) / distX)) * dtV;
    v->posZ += (velTramoX * ((-14.96f - -11.258f) / distX)) * dtV;
    v->anguloX += (velTramoX * ((89.988f - 86.84f) / distX)) * dtV;
    v->anguloY += (velTramoX * ((70.676f - 70.695f) / distX)) * dtV;
    v->anguloZ += (velTramoX * ((-0.21197f - 0.89183f) / distX)) * dtV;

    if (!v->deRegreso) {
        if (v->posX <= -154.47f) {
            v->posX = -154.47f; v->posY = 27.491f; v->posZ = -14.96f;
            v->anguloX = 89.988f; v->anguloY = 70.676f; v->anguloZ = -0.21197f;
            v->etapa = 16;
        }
    }
}
}

```

```

else
    if (v->posX >= -142.49f) {
        v->posX = -142.49f; v->posY = 27.339f; v->posZ = -11.258f;
        v->anguloX = 86.84f; v->anguloY = 70.695f; v->anguloZ = 0.89183f;
        v->etapa = 14;
    }
}

// --- ETAPA 16 ---
else if (v->etapa == 16) {
    distX = 11.38f;
    v->posX -= velTramoX * dtV;
    v->posY += (velTramoX * ((26.779f - 27.491f) / distX)) * dtV;
    v->posZ += (velTramoX * ((-18.545f - -14.96f) / distX)) * dtV;
    v->anguloX += (velTramoX * ((94.994f - 89.988f) / distX)) * dtV;
    v->anguloY += (velTramoX * ((70.722f - 70.676f) / distX)) * dtV;
    v->anguloZ += (velTramoX * ((2.3065f - -0.21197f) / distX)) * dtV;

    if (!v->deRegreso) {
        if (v->posX <= -165.85f) {
            v->posX = -165.85f; v->posY = 26.779f; v->posZ = -18.545f;
            v->anguloX = 94.994f; v->anguloY = 70.722f; v->anguloZ = 2.3065f;
            v->etapa = 17;
        }
    }
    else {
        if (v->posX >= -154.47f) {
            v->posX = -154.47f; v->posY = 27.491f; v->posZ = -14.96f;
            v->anguloX = 89.988f; v->anguloY = 70.676f; v->anguloZ = -0.21197f;
            v->etapa = 15;
        }
    }
}

// =====
// ETAPA 17: Final del recorrido y Lógica de Retorno
// =====
else if (v->etapa == 17) {
    distX = 11.61f;
    v->posX -= velTramoX * dtV;
    v->posY += (velTramoX * ((25.389f - 26.779f) / distX)) * dtV;
    v->posZ += (velTramoX * ((-23.568f - -18.545f) / distX)) * dtV;
    v->anguloX += (velTramoX * ((96.958f - 94.994f) / distX)) * dtV;
    v->anguloY += (velTramoX * ((70.654f - 70.722f) / distX)) * dtV;
    v->anguloZ += (velTramoX * ((1.6187f - 2.3065f) / distX)) * dtV;

    if (!v->deRegreso) {
        if (v->posX <= -177.46f) { // Llegamos al final absoluto de la pista
            v->posX = -177.46f; v->posY = 25.389f; v->posZ = -23.568f;
            v->anguloX = 96.958f; v->anguloY = 70.654f; v->anguloZ = 1.6187f;
            // --- DECISIÓN FINAL ---
            if (i_vehiculo == 1) {
                // Si es la Nave (índice 1), se queda pausada en la meta.
                v->termino = true;
            }
            else {
                // Si es el Coche (índice 0), se activa la reversa y empieza a retroceder.
                // NOTA: NO cambiamos de etapa, se queda en la 17 para recorrerla hacia atrás.
                v->deRegreso = true;
            }
        }
    }
    else {
        if (v->posX >= -165.85f) { // Origen de la etapa 17 (Para la vuelta del coche)
            v->posX = -165.85f; v->posY = 26.779f; v->posZ = -18.545f;
            v->anguloX = 94.994f; v->anguloY = 70.722f; v->anguloZ = 2.3065f;
            v->etapa = 16;
        }
    }
}
}
}

```

Window.h

Como manejamos el teclado, instanciamos getters para registrar eventos externos como lo hemos hecho en prácticas anteriores en nuestras secciones de private y public respectivamente

```
// --- ANIMACION COCHE ---
bool getReiniciaCocher() { return reiniciaCocher; }
void setReiniciaCocher(bool estado) { reiniciaCocher = estado; } // Para apagar la señal después de usarla
...
...
...
```

```
// --- NUEVAS VARIABLES ---  
bool reiniciaCoche;
```

Window.cpp

Además de inicializar reinicio coche en false al inicio de nuestro cpp, en la función maneja teclado declaramos el uso de M

```
// [M]: Reiniciar el recorrido del coche (Ida y Vuelta)  
if (key == GLFW_KEY_M && action == GLFW_PRESS) {  
    theWindow->reiniciaCoche = true;  
}
```

MAIN.cpp

Al inicio de el archivo main llamamos a nuestro archivo de cabecera para el recorrido.

```
#include "RecorridoP.h"  
RecorridoP animadorRuta; // Instanciamos la clase globalmente
```

Dentro de nuestra función int main declaramos nuestras variables que usaremos para algunas transformaciones y controladores extra, esto fuera de nuestro bucle por cada ciclo de reloj

```
// =====  
// DECLARACIONES MOVIDAS FUERA DEL CICLO  
// =====  
  
// --- 1. TIEMPO Y VELOCIDADES BASE ---  
GLfloat now = 0.0f;  
float velTramoX = 0.35f; // VELOCIDAD de la máquina de estados en el eje X  
float velTramoY = 0.35f; // VELOCIDAD proporcional en Y (Uso interno de etapa)  
float velTramoZ = 0.35f; // VELOCIDAD proporcional en Z (Uso interno de etapa)  
  
// --- 2. ANIMACIONES GLOBALES Y TEXTURAS ---  
toffsetflechau = 0.0f;  
toffsetflechav = 0.0f;  
toffsetnumerou = 0.0f;  
toffsetnumerov = 0.0f;  
toffsetnumerocambiau = 0.0f;  
angulovaria = 0.0f;  
rotllanta = 0.0f; // Ángulo actual de las llantas  
rotAleteo = 0.0f; // Ángulo actual de las alas de la nave  
aleteaSube = true; // Bandera de dirección del aleteo (arriba/abajo)  
limiteAleteo = 30.0f; // Ángulo máximo de apertura de las alas  
rotHelices = 0.0f; // Ángulo acumulado de las hélices  
rotllantaOffset = 2.0f; // VELOCIDAD DE ROTACIÓN DE LAS LLANTAS  
velAleteo = 0.5f; // VELOCIDAD DE ALETEO  
velHelices = 10.0f; // VELOCIDAD DE ROTACIÓN DE LAS HELICES  
flagReinicio = false; // Bandera para reiniciar animaciones al volver a la etapa 1  
  
// --- 3. VARIABLES BASE DE SHADER Y RENDERIZADO ---  
glm::mat4 model = glm::mat4(1.0f);  
glm::mat4 modelaux = glm::mat4(1.0f);  
glm::vec3 color = glm::vec3(1.0f, 1.0f, 1.0f);  
glm::vec2 toffset = glm::vec2(0.0f, 0.0f);  
glm::vec3 lowerLight = glm::vec3(0.0f, 0.0f, 0.0f);  
unsigned int lucesPuntualesActivas = 0;  
  
// --- 4. JERARQUÍA Y LUCES: COCHE ---  
glm::mat4 modelCompensado = glm::mat4(1.0f); // Matriz para corregir la rotación en reversa  
glm::mat4 matrizCocheLuz = glm::mat4(1.0f); // Matriz base anclada al coche para el faro  
glm::vec4 posicionLocalFaroCoche = glm::vec4(8.8f, 2.1f, 0.0f, 1.0f); // Posición del faro respecto al chasis  
glm::vec4 posFaroFrente = glm::vec4(0.0f); // Posición temporal del faro en el frame  
glm::vec3 dirFaroFrente = glm::vec3(0.0f); // Vector de dirección de la luz temporal  
glm::vec4 posicionGlobalFaroCoche = glm::vec4(0.0f); // Coordenada real del faro en el mundo 3D  
glm::vec3 dirFaroGlobal = glm::vec3(0.0f); // Hacia dónde apunta la luz en el mundo 3D
```

Para nuestro ciclo de reloj declaramos la animación pingpong con nuestra clase de RecorridoP

```
// 6. MÁQUINA DE ESTADOS UNIFICADA (Coche y Nave) -----  
// 6.1 Extraemos el valor del teclado desde la ventana
```

```

flagReinicio = mainWindow.getReiniciaCoche();
// 6.2 Pasamos el valor a la clase (por referencia) y ejecutamos las 17 etapas.
animadorRuta.actualizarRuta(deltaTime, velTramoX, flagReinicio);
// 6.3 Sincronización Inversa:
// Si la clase reseteó el coche, 'flagReinicio' habrá cambiado a false internamente.
// Si es así, actualizamos la ventana para apagar la señal del teclado.
if (!flagReinicio && mainWindow.getReiniciaCoche()) {
    mainWindow.setReiniciaCoche(false);
}

// 7. Rotación de llantas -----
// Consultamos la variable pública 'termino' de nuestra clase.
if (!animadorRuta.coche.termino) {
    rotllanta += rotllantaOffset * deltaTime;
}

```

Finalmente, actualizamos la posición de nuestro coche, su luz ligada y el renderizado del modelo.

```

// =====
// FASE 1: CÁLCULO DE MATRICES Y SUS JERARQUÍAS DE LUCES
// =====

// =====
// MATRIZ Y LUZ: COCHE (Espacio Local vs Espacio Global)
// =====
modelaux = glm::mat4(1.0f); // Iniciamos con una matriz "limpia" (Identidad) en el centro del mundo.
// Movemos el origen de la matriz a la posición actual del coche.
modelaux = glm::translate(modelaux, glm::vec3(animadorRuta.coche.posX, animadorRuta.coche.posY, animadorRuta.coche.posZ));
// Aplicamos las rotaciones de la pista (Em orden Y -> X -> Z)
modelaux = glm::rotate(modelaux, animadorRuta.coche.anguloY * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
modelaux = glm::rotate(modelaux, (90.0f - animadorRuta.coche.anguloX) * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
modelaux = glm::rotate(modelaux, -animadorRuta.coche.anguloZ * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
// Giramos el modelo -90° en Y para corregir cómo trabajamos desdeBlender
modelCompensado = glm::rotate(modelaux, -90.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
// Si el coche va de reversa, giramos todo el modelo 180° visualmente para que la cajuela apunte hacia adelante.
if (animadorRuta.coche.deRegreso) {
    modelCompensado = glm::rotate(modelCompensado, 180.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
}
// LUZ DEL FARO DEL COCHE (Spot Light)
spotLights[0] = farosAuxiliares[mainWindow.getColorFaroCoche()]; // Cargamos el color del faro
matrizCocheLuz = modelCompensado; // La luz hereda TODAS las transformaciones del coche
// Posición y Dirección "Locales"
posFaroFrente = glm::vec4(-7.5f, 2.1f, 0.0f, 1.0f); // El faro está físicamente adelante (-X) y un poco arriba (+Y).
dirFaroFrente = glm::vec3(-1.0f, -0.2f, 0.0f); // Apunta derecho hacia adelante (-X) y ligeramente hacia abajo (-Y).
// Multiplicamos Local * Matriz para obtener las coordenadas reales en el mundo 3D.
posicionGlobalFaroCoche = matrizCocheLuz * posFaroFrente;
// Para la dirección, usamos mat3 para ignorar la Traslación (no se puede "mover" una dirección, solo rotarla).
dirFaroGlobal = glm::mat3(matrizCocheLuz) * dirFaroFrente;
// Le enviamos los datos finales calculados al Shader.
spotLights[0].SetFlash(glm::vec3(posicionGlobalFaroCoche), dirFaroGlobal);

...
...
// =====
// FASE 3: RENDERIZADO DE MODELOS OPACOS
// =====
// COCHE -----
model = modelCompensado;
model = glm::rotate(model, 90.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f)); // Recupera su rotación visual interna
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Cochet.RenderModel();
color = glm::vec3(0.5f, 0.5f, 0.5f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
// Llanta trasera derecha
model = glm::translate(modelCompensado, glm::vec3(4.3f, 0.4f, 3.1f));
model = glm::rotate(model, rotllanta * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, -90.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
LlantaT.RenderModel();
// Llanta trasera izquierda
model = glm::translate(modelCompensado, glm::vec3(4.3f, 0.4f, -3.1f));
model = glm::rotate(model, rotllanta * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));

```

```

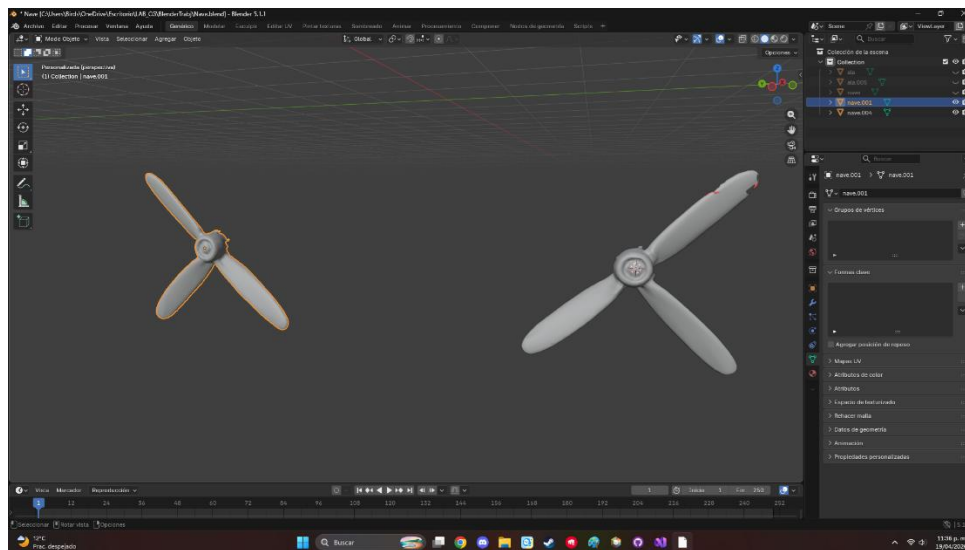
model = glm::rotate(model, 90.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
LlantaT.RenderModel();
// Llanta delantera derecha
model = glm::translate(modelCompensado, glm::vec3(-6.0f, 0.4f, 3.1f));
model = glm::rotate(model, rotllanta * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, -90.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
LlantaT.RenderModel();
// Llanta delantera izquierda
model = glm::translate(modelCompensado, glm::vec3(-6.0f, 0.4f, -3.1f));
model = glm::rotate(model, rotllanta * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, 90.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
LlantaT.RenderModel();
// COFRE
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
model = glm::translate(modelCompensado, glm::vec3(-4.2f, 2.6f, 0.0f));
model = glm::rotate(model, 90.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getAperturaCofre()), glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
CofreT.RenderModel();

```

RECORRIDO NAVE

INSTRUCCIÓN: “Separar de la Nave el otra ala y las dos hélices, Hacer que sobrevuele la pista adecuadamente (jerarquía, aleteo, giro de hélices, faro ilumina la pista) pero en sentido contrario, inicia en el punto extremo y aterriza junto al punto inicial. Esta animación no se puede repetir.”

Primero pasamos a Blender para separar las hélices que nos hacían falta, ambas alas ya habían quedado funcionales desde el ejercicio de clase.



Los únicos cambios para esta parte de las actividades fueron en el main, ya que el recorrido es el mismo del auto, la nave se instancia como otro usuario de la estructura vehículo que definimos en RecorridoP, siendo su diferencia de que su movimiento irá desde el fin de la pista hasta el inicio.

MAIN.cpp

Al inicio del archivo main declaramos variables globales para la animación de las alas y las hélices. También exportamos nuestro modelo de Nave, las dos alas y hélices

```
// --- Animación: Nave (Aleteo de alas en Z) ---
bool aleteaSube; // Estado del aleteo (true = rotando hacia arriba)
float rotAleteo; // Ángulo actual en el que se encuentra el ala
float velAleteo; // Qué tan rápido rotan las alas (grados por segundo)
float limiteAleteo; // Límite de rotación máxima (amplitud del aleteo)

// --- Animación: Nave (Hélices en Z) ---
float rotHelices; // Ángulo actual de las hélices
float velHelices; // Velocidad de giro
...
...
Model NaveT;
Model Ala1T;
Model Ala2T;
Model Helice1T;
Model Helice2T;
```

Dentro de nuestro int main cargamos los modelos, declaramos las variables de apoyo y las matrices para transformaciones

```
NaveT = Model(); NaveT.LoadModel("Models/NaveT.obj");
Ala1T = Model(); Ala1T.LoadModel("Models/Ala1T.obj");
Ala2T = Model(); Ala2T.LoadModel("Models/Ala2T.obj");
Helice1T = Model(); Helice1T.LoadModel("Models/Helice1T.obj");
Helice2T = Model(); Helice2T.LoadModel("Models/Helice2T.obj");
...
...
rotAleteo = 0.0f; // Ángulo actual de las alas de la nave
aleteaSube = true; // Bandera de dirección del aleteo (arriba/abajo)
limiteAleteo = 30.0f; // Ángulo máximo de apertura de las alas
rotHelices = 0.0f; // Ángulo acumulado de las hélices
rotLlantaOffset = 2.0f; // VELOCIDAD DE ROTACIÓN DE LAS LLANTAS
velAleteo = 0.5f; // VELOCIDAD DE ALETEO
velHelices = 10.0f; // VELOCIDAD DE ROTACIÓN DE LAS HELICES
...
...
// --- 5. JERARQUÍA Y LUCES: NAVE ---
glm::mat4 matrizBaseNave = glm::mat4(1.0f); // Matriz pura sacada de la máquina de estados
glm::mat4 matrizNaveVisual = glm::mat4(1.0f); // Matriz parcheada visualmente (compensación Blender)
glm::mat4 matrizNaveLuz = glm::mat4(1.0f); // Matriz de anclaje para la luz de la nave
glm::vec4 posLocalFaroNave = glm::vec4(-1.0f, -0.5f, 0.0f, 1.0f); // Donde va el foco en la nave
glm::vec4 posFaroNaveFrente = glm::vec4(0.0f); // Cálculo temporal de posición
glm::vec3 dirFaroNaveFrente = glm::vec3(0.0f); // Cálculo temporal de dirección
glm::vec4 posGlobalFaroNave = glm::vec4(0.0f); // Posición real del faro de la nave en el mundo
glm::vec3 dirLocalFaroNave = glm::vec3(0.0f); // Hacia dónde apunta internamente
glm::vec3 dirGlobalFaroNave = glm::vec3(0.0f); // Hacia dónde dispara la luz en el mundo
```

Dentro de nuestro bucle por cada ciclo de reloj aplicamos la animación de las alas, las hélices y el recorrido en reversa.

```
// =====
// LÓGICA DE ANIMACIÓN PING-PONG
// =====

// 4. Lógica Ping-Pong del Aleteo (Rotación en Z) -----
if (aleteaSube) {
    // Fase de subida: Sumamos velocidad hasta tocar el "techo"
    rotAleteo += velAleteo * deltaTime;
    if (rotAleteo >= limiteAleteo) {
        rotAleteo = limiteAleteo; // Anclamos al tope exacto
        aleteaSube = false; // Invertimos la bandera para que baje
    }
}
else {
    // Fase de bajada: Restamos velocidad hasta tocar el "piso"
    rotAleteo -= velAleteo * deltaTime;
    if (rotAleteo <= -limiteAleteo) {
```

```

        rotAleteo = -limiteAleteo; // Anclamos al piso exacto
        aleteaSube = true;          // Invertimos la bandera para que suba
    }
}

// 5. Lógica Ping-Pong de rotación continua (Hélices en Z) -----
// Al no tener límites, el ángulo crecerá por siempre, generando un giro constante.
rotHelices += velHelices * deltaTime;

// 6. MÁQUINA DE ESTADOS UNIFICADA (Coche y Nave) -----
// 6.1 Extraemos el valor del teclado desde la ventana
flagReinicio = mainWindow.getReiniciaCoche();
// 6.2 Pasamos el valor a la clase (por referencia) y ejecutamos las 17 etapas.
animadorRuta.actualizarRuta(deltaTime, velTramoX, flagReinicio);
// 6.3 Sincronización Inversa:
// Si la clase reseteó el coche, 'flagReinicio' habrá cambiado a false internamente.
// Si es así, actualizamos la ventana para apagar la señal del teclado.
if (!flagReinicio && mainWindow.getReiniciaCoche()) {
    mainWindow.setReiniciaCoche(false);
}
}

```

Finalmente, actualizamos la posición de nuestra nave, su luz ligada y el renderizado del modelo

```

// =====
// FASE 1: CÁLCULO DE MATRICES Y SUS JERARQUÍAS DE LUCES
// =====

// =====
// MATRIZ Y LUZ: NAVE (Espacio Local vs Espacio Global)
// =====
matrizBaseNave = glm::mat4(1.0f); // Iniciamos con una matriz "limpia" (Identidad) en el centro del mundo.
// Movemos el origen de la matriz a la posición actual de la nave, sumando 10 unidades en Y para que vuele.
matrizBaseNave = glm::translate(matrizBaseNave, glm::vec3(animadorRuta.nave.posX, animadorRuta.nave.posY + 10.0f, animadorRuta.nave.posZ));
// Aplicamos las rotaciones de la pista (En orden Y -> X -> Z)
matrizBaseNave = glm::rotate(matrizBaseNave, animadorRuta.nave.anguloY * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
matrizBaseNave = glm::rotate(matrizBaseNave, (90.0f - animadorRuta.nave.anguloX) * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
matrizBaseNave = glm::rotate(matrizBaseNave, -animadorRuta.nave.anguloZ * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
// Giramos el modelo 90° en Y para corregir cómo trabajamos desde Blender
matrizNaveVisual = glm::rotate(matrizBaseNave, 90.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
// LUZ DEL FARO DE LA NAVE (Spot Light)
spotLights[1] = farosAuxiliares[6]; // Cargamos el faro auxiliar amarillo hacia adelante
matrizNaveLuz = matrizNaveVisual; // La luz hereda TODAS las transformaciones de la nave
// Posición y Dirección "Locales"
posFaroNaveFrente = glm::vec4(-4.0f, -0.5f, 0.0f, 1.0f); // El faro está físicamente en la nariz (-X) y un poco abajo (-Y).
dirFaroNaveFrente = glm::vec3(-1.0f, -1.0f, 0.0f); // Apunta en diagonal hacia adelante (-X) y hacia el suelo (-Y).
// Multiplicamos Local * Matriz para obtener las coordenadas reales en el mundo 3D.
posGlobalFaroNave = matrizNaveLuz * posFaroNaveFrente;
// Para la dirección, usamos mat3 para ignorar la Traslación (no se puede "mover" una dirección, solo rotarla).
dirGlobalFaroNave = glm::mat3(matrizNaveLuz) * dirFaroNaveFrente;
// Le enviamos los datos finales calculados al Shader.
spotLights[1].SetFlash(glm::vec3(posGlobalFaroNave), dirGlobalFaroNave);

...
...
...
// NAVE -----
model = matrizNaveVisual; // ¡Usamos la matriz mágica calculada en la Fase 1!
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
NaveT.RenderModel();
// HÉLICE 1 (Costado Derecho)
model = matrizNaveVisual;
model = glm::translate(model, glm::vec3(-2.3f, -2.1f, -2.55f));
model = glm::rotate(model, rotHelices * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Helice1T.RenderModel();
// HÉLICE 2 (Costado Izquierdo)
model = matrizNaveVisual;
model = glm::translate(model, glm::vec3(-2.3f, -2.1f, 2.55f));
model = glm::rotate(model, rotHelices * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Helice2T.RenderModel();
// ALA 1 (Costado Derecho)

```

```

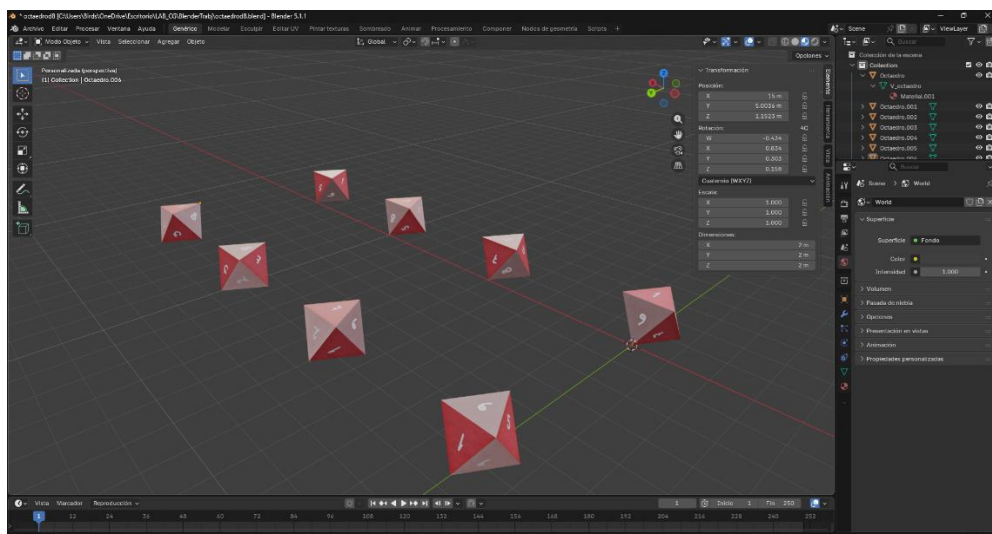
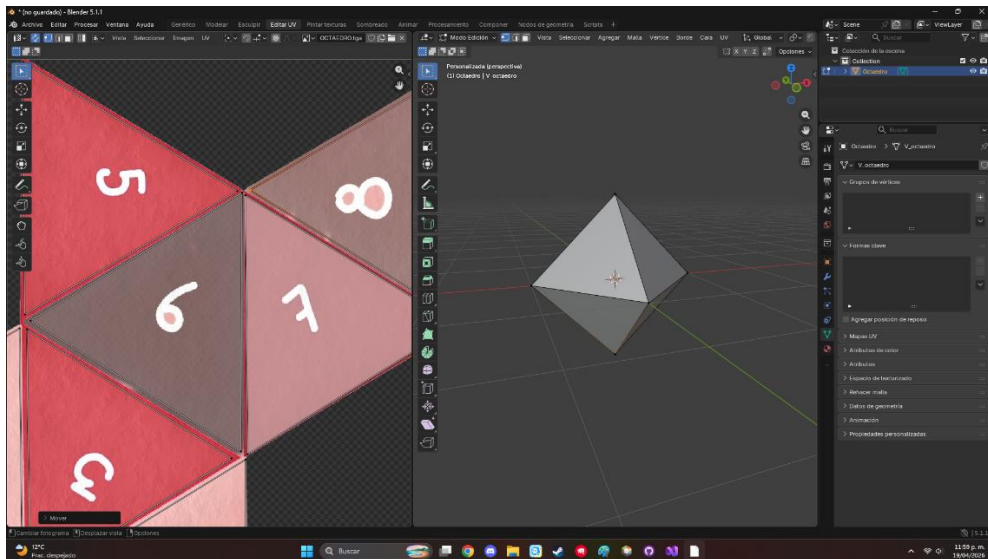
model = matrizNaveVisual;
model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.5f));
model = glm::rotate(model, rotAleteo * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Ala1T.RenderModel();
// ALA 2 (Costado Izquierdo)
model = matrizNaveVisual;
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -0.5f));
model = glm::rotate(model, -rotAleteo * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Ala2T.RenderModel();

```

TIRO DADO D8

INSTRUCCIÓN: “Su dado de 8 caras cae girando sobre el piso y muestra un número random *adecuadamente* hacia arriba, se repite la tirada al presionar una tecla.”

Primero descargamos un modelo de un octaedro desde [Sketchfab](#) y lo texturizamos con mapas UV en Blender. La intención era repetir el proceso del recorrido por la pista, definir los condicionales para cada caso de cara y después aplicar esas transformaciones con ayuda de Blender.



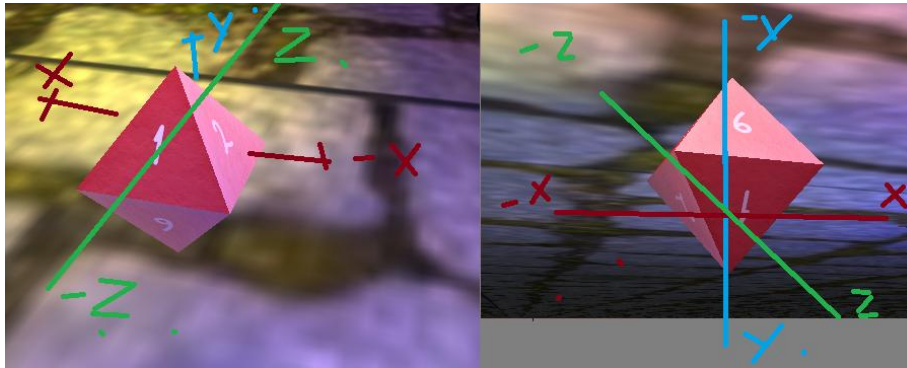
Sin embargo, no fue tan sencillo, ya que no habíamos definido primero la lógica de animación el mover el octaedro libremente en Blender solo ayudo a tener un calculo aproximado sobre que ejes y cuantos grados se debía rotar el dado para quedar en X cara.

Se decidió entonces ir por la siguiente opción: Se haría una animación donde el dado rotaría en todos los ejes varias veces en 360° para después usar la función rand que nos haría elegir entre un numero aleatorio y sus 8 posibles casos de resultado, dependiendo de este, se definiría la transformación resultante en base a un arreglo de vectores y la “meta” al terminar los giros del dado se definirían con el residuo de rand.

Se agrego la letra N en window.cpp para aplicar el tiro y se exporto el modelo del octaedro en nuestro archivo main y con esto en mente, se hicieron las siguientes modificaciones.

MAIN.cpp

Dentro de int main declaramos variables de apoyo fuera de nuestro bucle while, es importante mencionar que las metas se calcularon a prueba y error ya que solo eran 8 posibles casos a diferencia del recorrido por pista.



```
// =====
// DECLARACIONES MOVIDAS FUERA DEL CICLO
// =====
// --- 8. VARIABLES DE ANIMACIÓN: DADO D8 ---
bool dadoRodando = false; // ¿Dado en el aire (true) o en el piso (false)?
float tiempoDado = 0.0f; // Cronómetro interno que lleva la cuenta del tiro actual
float duracionTiro = 150.0f; // Tiempo total del vuelo
int resultadoDado = 0; // Resultado
// Para el tiro-vuelo del d8
float t = 0.0f; // Tiempo de rot
float tSuave = 0.0f;
float alturaDestino;
float baseYActual;

glm::vec3 posDadoBase = glm::vec3(0.0f, -2.0f, 10.0f); // Coordenada donde descansa el dado
glm::vec3 posDadoActual = posDadoBase; // Posición dinámica (Sube y baja durante la parábola)
glm::vec3 rotDadoActual = glm::vec3(0.0f); // Rotación dinámica actual en pantalla (X, Y, Z)
glm::vec3 rotDadoInicio = glm::vec3(0.0f); // Ángulo exacto desde el que arranca el tiro
glm::vec3 rotDadoDestino = glm::vec3(0.0f); // Ángulo meta al que debe llegar (Cara ganadora + Vueltas extra)

// Inicializamos la semilla aleatoria tomando el reloj de la computadora
srand(time(NULL));

// METAS DEL DADO D8 (Ángulos Euler en Grados)
glm::vec3 metasD8[8] = {
    glm::vec3(55.0f, 0.0f, 0.0f), // Cara 1
    glm::vec3(0.0f, -90.0f, -55.0f), // Cara 2
    glm::vec3(-55.0f, -180.0f, 0.0f), // Cara 3
    glm::vec3(0.0f, -270.0f, 55.0f), // Cara 4
    glm::vec3(180.0f, 90.0f, -55.0f), // Cara 5 (+1.2 en Y)
    glm::vec3(-235.0f, 0.0f, 0.0f), // Cara 6 (+1.2 en Y)
    glm::vec3(180.0f, 90.0f, 55.0f), // Cara 7 (+1.2 en Y)
```

```
}; glm::vec3(235.0f, 0.0f, 0.0f) // Cara 8 (+1.2 en Y)
```

Dentro de nuestro ciclo de renderizado establecemos la animación del dado. Al presionar la tecla 'N', el programa sorteá de forma aleatoria la cara que va a caer y calcula a qué ángulo exacto debe llegar (gracias a nuestro arreglo de vec3). Durante el tiempo que dura el “vuelo”, un cronómetro interno avanza del 0% al 100% utilizando una curva de 'frenado suave'; esto logra que el dado comience girando a máxima velocidad y vaya perdiendo fuerza poco a poco antes de aterrizar. Además verificamos que si el resultado está entre 5 y 8 subimos en Y 1.2 unidades.

```
// 8. Tiro D8 -----
// 8.1 Verificamos tecla (N)
if (mainWindow.getTiroD8() && !dadoRodando) {
    dadoRodando = true; // Cambiamos el estado a "En el aire"
    tiempoDado = 0.0f; // Reiniciamos el cronómetro de la animación
    mainWindow.setTiroD8(false); // Apagamos la señal del teclado
    // 1. Aleatoriamente decidimos resultado con un residuo
    resultadoDado = rand() % 8;
    // 2. Guardamos la posición
    rotDadoInicio = rotDadoActual;
    // 3. Para el destino: Tomamos la meta y le sumamos vueltas completas (360°).
    rotDadoDestino = metasD8[resultadoDado] + glm::vec3(360.0f * 4, 360.0f * 6, 360.0f * 3);
    // Impresión para verificar resultado esperado
    printf("El dado caera en la cara: %d\n", resultadoDado + 1);
}
// Ignoramos la tecla si el dado ya está a medio vuelo para evitar que se reinicie de golpe
else if (mainWindow.getTiroD8() && dadoRodando) {
    mainWindow.setTiroD8(false);
}

// 8.2 Animación de tiro
if (dadoRodando) {
    tiempoDado += deltaTime; // Avanzamos el reloj interno

    // Convertimos el tiempo a un porcentaje que va de 0.0 (inicio) a 1.0 (final).
    t = tiempoDado / duracionTiro;
    if (t > 1.0f) t = 1.0f; // Para que no pase del 100%

    // CURVA DE DESACELERACIÓN (Fórmula: Ease-Out Cubic)
    // Altera la variable 't' para que arranque a máxima velocidad y frene suavemente al final.
    tSuave = 1.0f - pow(1.0f - t, 3.0f);
    // ROTACIÓN: INSERTANDO DE MANERA LINEAL (Lerp)
    // Mezclamos el ángulo inicial con el destino calculados en el Paso A, usando el tiempo suavizado.
    rotDadoActual = glm::mix(rotDadoInicio, rotDadoDestino, tSuave);

    // Revisamos si el resultado requiere +1.2 Y
    alturaDestino = posDadoBase.y;
    if (resultadoDado >= 4) {
        alturaDestino += 1.2f;
    }
    // Insertamos a nivel de piso
    baseYActual = glm::mix(posDadoBase.y, alturaDestino, tSuave);

    // Calculamos el salto aplicando la función Seno y se lo sumamos al piso móvil.
    // (El seno va de 0 a 1 y regresa a 0, creando una parábola perfecta de altura 3.0).
    posDadoActual.y = baseYActual + sin(t * 3.14159f) * 3.0f;

    // ATERRIZAJE
    if (tiempoDado >= duracionTiro) {
        dadoRodando = false; // 'Apagamos' animación
        // Forzamos la altura final
        if (resultadoDado >= 4) posDadoActual.y = posDadoBase.y + 1.2f;
        else posDadoActual.y = posDadoBase.y;
        // Pasamos de rotación actual por la meta
        rotDadoActual = metasD8[resultadoDado];
    }
}
```

Finalmente, renderizamos nuestro modelo en la sección de modelos con transparencia.

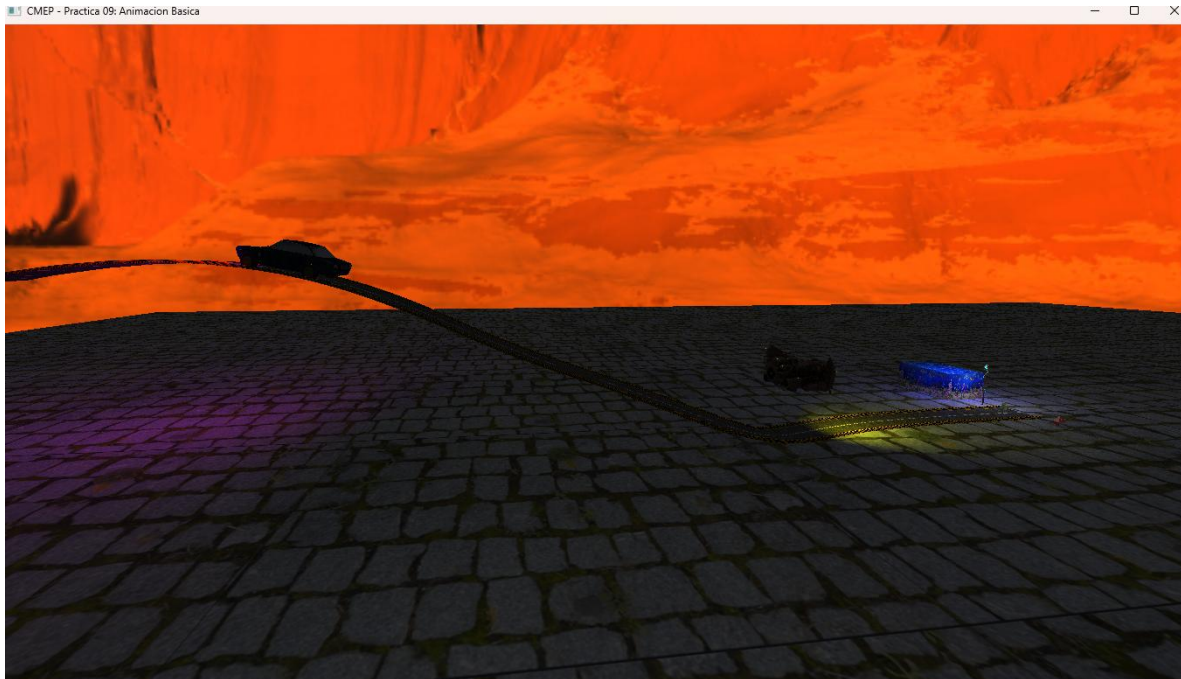
```
// DADO D8 COMPENSADO -----
model = glm::mat4(1.0f);
// Lo movemos a su posición física + la compensación (si la hay)
model = glm::translate(model, glm::vec3(posDadoActual.x, posDadoActual.y, posDadoActual.z));
// Aplicamos la rotación de la meta (o el caos si está rodando)
```



```
model = glm::rotate(model, rotDadoActual.y * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, rotDadoActual.x * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, rotDadoActual.z * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
D8T.RenderModel();
```

EJECUCIÓN

Las únicas teclas que usaremos serán ‘M’ para repetir el recorrido del coche una vez regresa al origen y ‘N’ para tirar el dado.



Se adjuntan links de drive a cada uno de los GIFS de resultado al igual que un .ZIP con los GIFS resultantes en la entrega por classroom

[LINK DE DESCARGA DEL .ZIP \(SE RECOMIENDA VER EL RESULTADO ASÍ\)](#)

https://drive.google.com/file/d/1vggFT0GPTx2B_1A0oVEXocJwnzGebSI2/view?usp=sharing

- Recorrido Coche: <https://drive.google.com/file/d/1yCO-QUxgrMtZLEct5rGRgKDe5hVyKHVX/view?usp=sharing>
- Recorrido Nave: https://drive.google.com/file/d/15QXSRCqtmKljKIT_9tfTZnz0Asbb7Nu/view?usp=sharing
- Tiro D8: <https://drive.google.com/file/d/1zR6dr5e118oh2ceqyT7jf7ipAg-vKJBy/view?usp=sharing>

PROBLEMAS PRESENTADOS

Durante el desarrollo e integración de los ejercicios de animación básica, se presentaron varios problemas, en especial con la manipulación de matrices, la física de la animación y la arquitectura del código. A continuación, se detallan los problemas y cómo fueron resueltos:

1. Desfase de ejes en la importación de modelos (El coche rotado 90°)

- **Problema:** Al cargar el modelo .obj del coche en OpenGL y aplicarle la matriz de traslación de la pista, el modelo aparecía rotado 90 grados sobre su eje Y, moviéndose "de lado" en lugar de apuntar hacia el frente de la trayectoria.
- **Solución:** Se implementó una "matriz de compensación" (modelCompensado). Antes de renderizar el modelo, se aplicó una rotación fija de -90.0f grados en el eje Y global después de calcular la posición y rotación de la ruta. Esto alineó el "frente" visual del modelo de Blender con el vector de dirección real de OpenGL.

2. Traducción de sistemas de coordenadas (Blender a OpenGL)

- **Problema:** Los motores gráficos manejan el espacio 3D de forma distinta. Blender utilizaba el eje Z como el vertical mientras que OpenGL utiliza el eje Y. Intentar vaciar las coordenadas y ángulos directamente al código generaba movimientos erráticos y posiciones incorrectas en el escenario.
- **Solución:** Se estableció una regla estricta de mapeo espacial: el eje X se mantuvo igual, la profundidad (eje Y en Blender) se invirtió y se asignó al eje Z de OpenGL, y la altura (eje Z de Blender) se asignó al eje Y de OpenGL. Con esta conversión en mente, se calcularon las distancias absolutas (distX) para alimentar el algoritmo de interpolación lineal.

3. Implementación del recorrido en reversa sin duplicar código

- **Problema:** Programar el viaje de regreso del coche implicaba, en teoría, reescribir la lógica matemática de las 17 etapas invirtiendo signos y direcciones, lo cual haría el código largo, ineficiente y difícil de mantener.
- **Solución (Asistida por IA):** Se utilizó Inteligencia Artificial para buscar un enfoque de optimización matemática. La IA sugirió implementar un operador negativo para invertir el flujo del tiempo ($dtV = v \rightarrow deRegreso ? -deltaTime : deltaTime;$). Al multiplicar las fórmulas por un tiempo negativo, el algoritmo recorrió las mismas ecuaciones hacia atrás automáticamente, resolviendo el problema con una sola línea de código.

4. Definición de metas para las rotaciones del Dado D8

- **Problema:** Intentar calcular los ángulos exactos para que las 8 caras del octaedro cayeran perfectamente planas sobre el suelo resultaba propenso a errores y colisiones con el piso.
- **Solución:** Se optó por una aproximación visual controlada. Las rotaciones se ajustaron manualmente en Blender en formato "XYZ Euler" hasta que cada cara quedó perfectamente paralela al suelo. Estos valores se registraron en el código dentro del arreglo metasD8.

Adicionalmente, se programó una compensación de +1.2f en el eje Y exclusiva para las caras 5 a 8, evitando que la mitad inferior del modelo atravesara el piso debido al centro de masa del octaedro.

5. Transición abrupta en la animación del Dado

- **Problema:** La primera iteración del tiro del dado utilizaba una suma constante de rotación que se detenía de golpe al agotarse el tiempo. El resultado visual era robótico, antinatural y carecía de peso físico.
- **Solución (Asistida por IA):** Se consultó a la Inteligencia Artificial para mejorar el tiro. La solución fue pre-calcular el resultado desde el inicio e implementar una curva de desaceleración matemática conocida como *Ease-Out Cubic* ($t_{\text{Suave}} = 1.0f - \text{pow}(1.0f - t, 3.0f)$). Esta fórmula modificó la variable de tiempo para que la función de mezcla (`glm::mix`) iniciara la rotación a máxima velocidad y aplicara un frenado suave y realista antes de aterrizar.

6. Saturación del archivo principal y falta de encapsulamiento

- **Problema:** Al inicio, toda la lógica de la máquina de estados, el control de las 17 etapas y las variables de los vehículos estaban amontonadas dentro del archivo `P09-317314975.cpp`, lo que volvía el código muy largo y difícil de leer o modificar. No se tenía claro cómo separar una lógica tan dependiente de las variables globales.
- **Solución (Asistida por IA):** Se solicitó apoyo a la IA para reestructurar el proyecto. Con su guía, se extrajo la lógica a un nuevo archivo (`RecorridoP.h` y `.cpp`), se agruparon las variables espaciales en una estructura `Vehiculo`, y se encapsuló el motor de estados dentro de una clase. Además, se aprendió a utilizar el paso de parámetros por referencia (`bool& reiniciaCoche`) para mantener la comunicación entre la lectura del teclado en la ventana y el reinicio interno del coche en la nueva clase.

APOYO DE IA

Implementación del recorrido en reversa sin duplicar código

Hola. Estoy haciendo una animación en OpenGL en C++ donde mi coche avanza por 17 etapas en una pista. Para moverlo de forma fluida siempre multiplico mi velocidad por el deltaTime. El problema es que ahora quiero que cuando llegue al final de la pista, recorra exactamente esas mismas 17 etapas pero en reversa. ¿Tengo que copiar y pegar todo mi código gigante de las 17 etapas y cambiar a mano todas las sumas por restas? Siento que es mucho código repetido. ¿Hay algún truco para engañar a mi programa y que el coche vaya en reversa usando exactamente el mismo bloque de código de ida?

¡Buenas noticias! No tienes que copiar y pegar ni una sola línea de lógica. En programación de gráficos y juegos, existe un concepto muy útil llamado **dirección** o **factor de escala temporal**.

En lugar de cambiar las operaciones matemáticas de suma a resta, lo que vamos a hacer es "engañar" al código manipulando el valor del `deltaTime` o creando una variable de dirección.

Aquí tienes tres formas elegantes de lograrlo sin repetir código:

1. El truco del Multiplicador de Dirección

Esta es la forma más limpia. Crea una variable llamada `direction` que valga `1.0` cuando vayas hacia adelante y `-1.0` cuando vayas hacia atrás.

```
C++  
  
float direction = 1.0f; // 1.0 = adelante, -1.0 = reversa  
  
// En tu bucle de actualización:  
if (llegoAlFinal) direction = -1.0f;  
if (llegoAlInicio) direction = 1.0f;  
  
// Tu código gigante de las 17 etapas se queda EXACTAMENTE igual:  
posicion += velocidad * deltaTime * direction;
```

¿Por qué funciona?

Matemáticamente, sumar un número negativo es lo mismo que restar. Al multiplicar por `-1.0`, todas tus sumas internas se convierten automáticamente en restas y los límites de tus etapas se invertirán lógicamente.

2. Invertir el DeltaTime (El truco del "Tiempo Negativo")

Si tu bloque de código es muy complejo y depende enteramente del `deltaTime`, puedes simplemente pasarle un `deltaTime` negativo cuando quieras que retroceda.

```
C++  
  
float dtEfectivo = deltaTime;  
  
if (modoReversa) {  
    dtEfectivo = -deltaTime;  
}  
  
// Llamas a tu función de las 17 etapas usando dtEfectivo  
actualizarPista(dtEfectivo);
```

***Nota:** Ten cuidado si usas funciones físicas muy complejas que no toleren valores de tiempo negativos (como algunas simulaciones de colisiones), pero para movimiento lineal funciona de maravilla.*

3. Uso de un "Ping-Pong" en el Factor de Progreso

Si tu animación depende de un valor de "progreso" (por ejemplo, una variable `t` que va de `0` a `17`), puedes usar la función `abs()` o un seno para crear un efecto de ida y vuelta.

Sin embargo, para tu caso específico de 17 etapas, la **Opción 1** es la más robusta porque mantiene la precisión de la velocidad constante.

***Nota:** Ten cuidado si usas funciones físicas muy complejas que no toleren valores de tiempo negativos (como algunas simulaciones de colisiones), pero para movimiento lineal funciona de maravilla.*

3. Uso de un "Ping-Pong" en el Factor de Progreso

Si tu animación depende de un valor de "progreso" (por ejemplo, una variable `t` que va de `0` a `17`), puedes usar la función `abs()` o un seno para crear un efecto de ida y vuelta.

Sin embargo, para tu caso específico de 17 etapas, la **Opción 1** es la más robusta porque mantiene la precisión de la velocidad constante.

Recomendación de Estructura

Para que no tengas que gestionar el cambio de etapa manualmente, asegúrate de que tus condiciones de "Etapa Completada" también sean reversibles:

- **Hacia adelante:** Si `posicion > limiteEtapa`, pasa a la siguiente.
- **En reversa:** Si `posicion < limiteEtapaAnterior`, pasa a la anterior.

Resumen

No toques las sumas. Solo añade una variable `float factor = 1.0f;` y multiplícala por tu movimiento. Cuando el coche toque el final de la etapa 17, cambia `factor = -1.0f;` y verás cómo el coche desanda el camino mágicamente usando el mismo bloque de código.

¿Cómo estás gestionando actualmente el cambio entre esas 17 etapas?

Transición abrupta en la animación del Dado

"Estoy animando un dado 3D en C++. Ahorita lo que hago es sumarle muchos grados a sus ejes XYZ todo el tiempo en mi ciclo while para que gire súper rápido, y cuando se acaba mi temporizador (unos 3 segundos), lo detengo de golpe en la posición de la cara que ganó. Pero se ve muy robótico e irreal porque se frena de la nada. ¿Cómo le hago matemáticamente para que el giro se vea natural? Quiero que empiece muy rápido, vaya perdiendo fuerza poco a poco en el aire, y aterrice súper suave. No sé mucho de curvas de animación, busco la forma más profesional de hacerlo."

Lo que buscas es pasar de un movimiento lineal (velocidad constante) a una **curva de desaceleración** (easing). En el desarrollo de videojuegos y animación profesional, esto se conoce comúnmente como **Ease-Out**.

Para que el dado se sienta real, no necesitas simular física compleja de colisiones, sino usar una función matemática que controle la velocidad basándose en el tiempo.

Aquí tienes la forma más profesional y estándar de implementarlo:

1. El concepto: Interpolación y Easing

En lugar de sumar grados infinitamente, vamos a definir un ángulo inicial ($A_{inicial}$) y un ángulo final ($A_{objetivo}$). El ángulo final debe ser el valor exacto de la cara ganadora (por ejemplo, 90° , 180° , etc.).

La fórmula mágica de **Ease-Out Quadratic** es:

$$f(t) = 1 - (1 - t)^2$$

Donde t es un valor de **0.0 a 1.0** que representa el progreso de tu temporizador.

2. Implementación en C++

Necesitas normalizar tu tiempo. Si tu animación dura 3 segundos, divide el tiempo transcurrido entre el tiempo total.

```
C++

float tiempoTotal = 3.0f;
float tiempoTranscurrido = 0.0f; // Incrementar con deltaTime cada frame

// 1. Normalizar el tiempo (t va de 0 a 1)
float t = tiempoTranscurrido / tiempoTotal;
if (t > 1.0f) t = 1.0f;

// 2. Aplicar la curva de Ease-Out (Quadratic o Cubic)
// Esto hace que 't' avance rápido al inicio y lento al final
float factorSuave = 1.0f - pow(1.0f - t, 3); // Usamos cubo para más suavidad

// 3. Interpolación la rotación
// AnguloActual = Inicio + (Objetivo - Inicio) * factorSuave
float rotacionActual = rotacionInicial + (rotacionObjetivo - rotacionInicial) * t
```

3. ¿Cómo calcular el `rotacionObjetivo`?

Para que el dado gire "muchas veces" antes de frenar, tu `rotacionObjetivo` no debe ser solo 90° , sino muchas vueltas completas más el ángulo de la cara final:

- **Vueltas deseadas:** 5 vueltas ($360 \times 5 = 1800^\circ$).
- **Cara final:** Supongamos que la cara ganadora está a 90° .
- **rotacionObjetivo:** $1800 + 90 = 1890^\circ$.

De esta forma, el código de arriba hará que el dado recorra esos 1890° empezando a toda velocidad y frenando suavemente justo en la cara que quieres.

4. Niveles de "Suavidad"

Dependiendo de la potencia que uses en la fórmula, el frenado será más o menos agresivo:

Tipo	Fórmula (t es el progreso)	Sensación
Quadratic	$1 - (1 - t)^2$	Frenado natural estándar.
Cubic	$1 - (1 - t)^3$	Más rápido al inicio, más lento al final.
Quartic	$1 - (1 - t)^4$	Frenado muy dramático y elegante.

📄 Exportar a Hojas de cálculo

Un truco extra: "The Overshoot" (Efecto Rebote)

Si quieres que el dado parezca que "aterriza" y rebota un poquito por el impacto, puedes usar esta fórmula (Back Ease-Out):

$$f(t) = 1 + 2.70158 \times (t - 1)^3 + 1.70158 \times (t - 1)^2$$

Esto hará que el dado se pase un poquito del ángulo final y regrese suavemente, dándole un peso físico increíble.

¿Te gustaría que te ayude a calcular los ángulos exactos de las 6 caras de tu dado para que encajen con esta fórmula?

Saturación del archivo principal y falta de encapsulamiento

```
float angulo;
int etapaActual;
bool enReversa;

public:
    Vehiculo(float inicioX, float inicioY, float inicioZ); // Constructor

    // La función estrella: Aquí metes tus 17 etapas
    void actualizar(float deltaTime, bool teclaAvanzar);

    // Función para que el main sepa dónde dibujar
    void dibujar();
};
```

2. ¿Cómo pasarle el deltaTime y las teclas?

El truco es que la función `actualizar` reciba esos valores como **parámetros**. No necesitas variables globales; el `main` le "pasa la información" al objeto en cada frame.

Vehiculo.cpp

```
C++

void Vehiculo::actualizar(float deltaTime, bool teclaAvanzar) {
    if (teclaAvanzar) {
        // Aquí metes tu bloque gigante de if/else de las 17 etapas
        // Pero ahora usas las variables internas (this->x, etc.)
        this->x += velocidad * deltaTime;

        if (etapaActual == 17) {
            // Lógica de reversa que vimos antes
        }
    }
}
```

Necesito ayuda organizando mi proyecto de C++ (OpenGL). Mi archivo main.cpp es un desastre. Tengo todas las variables globales de mi coche y mi nave (posiciones en X Y Z, ángulos, número de etapa, banderas) regadas por todos lados. Además, tengo un bloque gigante de if/else con 17 etapas metido justo a la mitad de mi ciclo while de dibujo. ¿Cómo puedo sacar toda esta lógica de animación y variables a otro archivo para limpiar mi main? Sé que debería usar Programación Orientada a Objetos (Clases), pero no tengo idea de cómo estructurarlo para que mis dos vehículos vivan ahí ni cómo pasarle el deltaTime o las teclas para que siga funcionando igual

Sacar la lógica de animación del `main.cpp` es el paso que separa a un programador de "scripts" de un desarrollador de motores gráficos. Tu `main` debería ser solo el director de orquesta, no el que toca todos los instrumentos.

Para limpiar ese desastre, vamos a aplicar un patrón de diseño llamado **Encapsulamiento**. Aquí tienes la estructura profesional para lograrlo:

1. La Estructura de la Clase (El "Molde")

Crea un archivo llamado `Vehiculo.h` (el encabezado) y `Vehiculo.cpp` (la lógica). La clave aquí es que cada vehículo guarde su propio estado.

Vehiculo.h

```
C++

class Vehiculo {
private:
    // Todas las variables que antes tenias regadas
    float x, y, z;
    float angulo;
```

3. Organizando el main.cpp

Ahora tu `main` se vuelve extremadamente limpio. Solo declaras tus objetos y los llamas dentro del bucle de OpenGL.

```
C++

// Variables globales mínimas (o dentro de una clase Engine)
Vehiculo coche(0.0f, 0.0f, 0.0f);
Vehiculo nave(10.0f, 5.0f, 0.0f);

// Dentro del ciclo while(windowOpen)
float dt = calcularDeltaTime();
bool moviendo = revisarTeclado();

// Actualizar lógica
coche.actualizar(dt, moviendo);
nave.actualizar(dt, moviendo);

// Dibujar
coche.dibujar();
nave.dibujar();
```

Beneficios de este orden:

- Independencia:** Si quieres 10 coches, solo creas 10 instancias de la clase `Vehiculo`. No tienes que duplicar variables *X, Y, Z* diez veces.
- Modularidad:** Si el coche tiene un error en la etapa 5, vas directo a `Vehiculo.cpp`, no tienes que buscar entre 2000 líneas del `main`.
- Abstracción:** Al `main` no le importa cómo se mueve el coche por las 17 etapas; solo le importa decirle "actualízate" y "dibújate".

El truco para las 17 etapas

Para limpiar el bloque gigante de `if/else`, te recomiendo usar un **State Machine** (Máquina de Estados) o un `switch(etapaActual)`. Si cada etapa tiene una meta (una coordenada X, por ejemplo), puedes guardar esas metas en un `std::vector<float>` y simplemente comparar:

```
C++

if (x > metas[etapaActual]) {
    etapaActual++;
}
```

¿Te hace sentido esta estructura o hay alguna parte del "paso de parámetros" que te genere dudas?

CONCLUSIONES

Sobre los ejercicios del reporte

El desarrollo de los ejercicios para esta práctica fue bastante más complejo a lo que percibí a diferencia de actividades anteriores. La transición de animaciones estáticas basadas en banderas simples hacia sistemas dinámicos controlados por **máquinas de estados finitas** termino por requerir un análisis mucho más profundo. Por este motivo, el enfoque de las explicaciones a lo largo de nuestro reporte se centró en desglosar detalladamente la lógica detrás de cada bloque de código (como el uso de `deltaTime`, las proporciones de avance y las curvas de desaceleración). El objetivo fue demostrar claramente que el movimiento tridimensional y la animación básica en OpenGL no son producto del azar, sino del cálculo y de una correcta arquitectura de software.

Comentarios generales

A nivel general, el desarrollo de esta práctica fue un gran apoyo para repasar transformaciones. Considero que en las sesiones hizo falta explicar a mayor detalle la base matemática quizá para que la animación no se sintiera tan robótica, particularmente el proceso de traducción de coordenadas de diseño (Blender) al motor gráfico (OpenGL) y el manejo de los Ángulos de Euler.

Conclusión

En conclusión, la realización de esta práctica nos ayudó a mejorar de forma práctica nuestros conocimientos para ejecuciones interactivas. Tener problemas como el desfase de ejes, la acumulación de errores de punto flotante en los límites de trayectoria y la necesidad de optimizar bloques masivos de código repetitivo nos obligó a buscar otras alternativas de implementación, como la alteración matemática del ciclo de tiempo (`-deltaTime`) o el pre-cálculo de animaciones físicas. Más allá de lograr que los modelos 3D se movieran por la pista, esta práctica nos permitió comprender cómo abstraer un problema físico complejo en su totalidad, encapsularlo de manera ordenada y modelarlo con precisión.

REFERENCIAS

- Gregory, J. (2018). *Game Engine Architecture* (3rd ed.). CRC Press.
- Lengyel, E. (2012). *Mathematics for 3D Game Programming and Computer Graphics* (3rd ed.). Course Technology.
- Nystrom, R. (2014). *Game Programming Patterns*. Genever Benning.
- Sellers, G., Wright, R. S., & Haemel, N. (2015). *OpenGL SuperBible: Comprehensive Tutorial and Reference* (7th ed.). Addison-Wesley Professional.
- Citizen. (s.f.). *Octaedro* [Modelo 3D]. Sketchfab. <https://sketchfab.com/3d-models/octaedro-de9a7d8243444002912e2b242195c025>
- de Vries, J. (s.f.). *LearnOpenGL - Transformations*. <https://learnopengl.com/Getting-started/Transformations>
- Quílez, I. (s.f.). *Useful little functions*. <https://iquilezles.org/articles/functions/>
- GameDev.net. (s.f.). *Understanding Delta Time*. <https://www.gamedev.net/articles/programming/general-and-gameplay-programming/understanding-delta-time-r5442/>